

Where do videos go to die?

Speaker script for the research update, October 5, 2026

Wun Ting Chan · 24 slides · about 24 minutes

The arc in one paragraph. We open with a striking picture: long AI videos all die the same death. We turn that into a question anyone can follow: where do long videos go, how fast, and which design choice decides? Then we explain how we got here. A year of trying to fix the decay failed in an instructive way, and that failure is why we switched to measuring it. The rest of the talk builds the case one piece at a time, each slide answering the doubt the previous one raises. Is this real? (Real video doesn't do it.) Is it one phenomenon? (No, three distinct fates.) Can we measure it? (Yes: one global attractor, a one-minute memory.) Is it a property of the weights? (No: one knob flips it.) Can you see it coming? (No, it's a ramp.) We close with what is new, stated carefully against prior measurement work, and what comes next.

[Bracketed grey text is stage direction: where to point, when to pause.]

Slide 1 Title



Good afternoon. The title is a little dramatic on purpose: where do videos go to die?

Over the next twenty minutes I want to convince you of three things. First, long AI-generated videos don't just slowly get worse. They fall into specific end states. Second, we can measure those end states with tools borrowed from physics. Third, which end state you get is decided by surprisingly small design choices.

This project began life as a test-time adaptation project. Part of this talk is the story of why it became something else.

Slide 2 AI can now generate video that never ends

The question
0000

How far can we go?
00

Algorithm
00

Results
000000000000000000

Next
0

AI can now generate video that never ends

► New *streaming* video models write a video one short chunk at a time, each chunk looking at the last few.

► In principle they can run forever: games, live worlds, endless scenes.

► We ran three of them for **3 minutes** on 8 very different prompts.

Tokyo street

Woolly mammoths

Space-man trailer

Big Sur waves

Fluffy monster

Paper coral reef

Crowned pigeon

Pirate ships in coffee

The first frame of each of the 8 prompts (Self Forcing).

What happens to a video if you just keep going?

Where do videos go to die? Win-Ting Chan 2

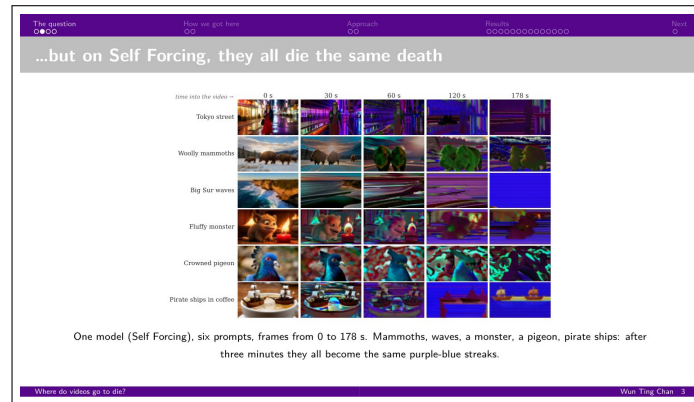
Point of this slide: set up the promise before showing the failure.

Here's the setting. A new family of video models, called streaming or autoregressive models, doesn't generate a whole clip at once. It writes a few frames, looks back at what it just wrote, and writes the next few. In principle that loop never has to stop. This is the technology behind live interactive worlds and game-like video.

[point at the grid] These are the opening frames from one of those models, given eight very different prompts: a woman on a Tokyo street, woolly mammoths, a sci-fi trailer, waves at Big Sur, a fluffy cartoon monster, a paper coral reef, a crowned pigeon, and pirate ships fighting in a cup of coffee. The openings look great. They're varied, detailed and on-prompt.

So the obvious experiment is: what happens if we just keep going? We let three of these models run for three full minutes, roughly 2,900 frames each.

Slide 3 ...but on Self Forcing, they all die the same death



Point of this slide: the hook. Let the image do the work.

[pause, let people look for a few seconds]

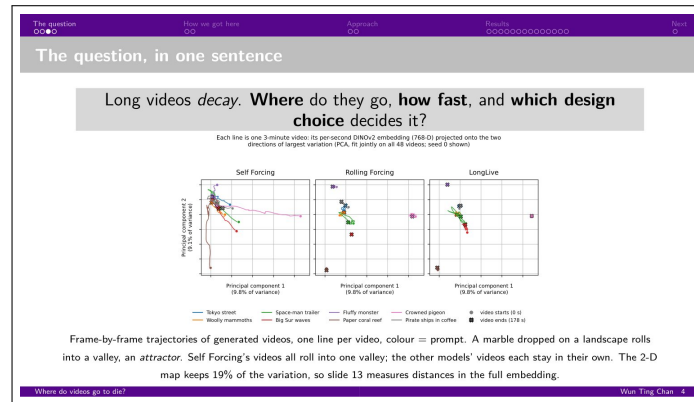
Each row is one prompt. Time runs left to right, from the first frame to three minutes. Read across any row and you watch a video decay. Now read down the last column.

The Tokyo street, the mammoths, the coastline, the monster, the pigeon, the pirate ships: by 178 seconds they've all turned into the same thing, purple and blue horizontal streaks. Six unrelated prompts arrive at one destination.

That's what I want you to remember. The decay isn't random noise piling up. It goes somewhere specific. For this model, everything goes to the same place.

Notice also that it isn't sudden. At 30 seconds you can already see the colour shift into neon purple. By 60 seconds the textures are breaking up. By 120 the content is mostly gone.

Slide 4 The question, in one sentence



Point of this slide: give the audience the mental model (marble and valley) before any technical vocabulary.

So here is the research question in one sentence. Long videos decay: where do they go, how fast, and which design choice decides it?

The picture to keep in mind is a marble on a hilly landscape. Wherever you drop it, it rolls downhill and settles in a valley. Physicists call that valley an attractor: a state the system is pulled toward and stays in.

[point at the three panels] This plot is our first evidence that the analogy is more than a metaphor. Its job is to show, in one picture, that the three models have differently shaped landscapes. Here is how to read it. For every frame of a video, an image encoder called DINOv2 gives a list of 768 numbers describing what's in the frame. We average those once per second, so a three-minute video becomes a path through that 768-dimensional space. To draw it, we find the two directions along which all the frames, from all 48 videos, vary the most. Those are the two principal components on the axes, and we project each path onto them. So each coloured line is the frame-by-frame trajectory of one generated video: the dot is its first frame, the cross is its last, and the colour tells you the prompt.

On the left, Self Forcing: the lines start all over the map and converge into one corner. That's a single valley everyone falls into. In the middle and on the right, Rolling Forcing and LongLive: each video barely moves from where it started. Every prompt has its own little valley.

One honest caveat, which is also in the caption: squashing 768 dimensions into two keeps only about 19% of the variation, so this map is a sketch. The actual claim rests on distances measured in the full space, which you'll see on slide 13.

Slide 5 Why this matters

The question
How we got here
Algorithm
Results
Next

Why this matters

A common problem, with very different outcomes
Streaming video, game engines and world models generate one step at a time from their own output. Long rollouts often accumulate drift, and recent designs (sinks, long-video tuning) reduce it a lot. In our runs, how a video ends depends strongly on the model and its inference settings.

Measured as symptoms, not as dynamics
Benchmarks and recent papers already quantify degradation: quality and consistency over long videos (VBench-Long), colour and motion drift, spectral drift, representation collapse, sink collapse. What we have not found is a characterization of the *dynamics*: where trajectories converge, how fast the prompt is lost, how perturbations grow, which design change alters the end state.

What we provide

- ▶ A map of each model's **end states**
- ▶ A **memory half-life** for the prompt
- ▶ **Sensitivity**: does a tiny nudge change the story?
- ▶ **Tipping knobs**: one design change flips the end state
- ▶ Tools anyone can run on a new model

How do videos go to die? What's going on? 5

Point of this slide: place the work precisely: a common problem with different outcomes, already measured as symptoms, not yet as dynamics.

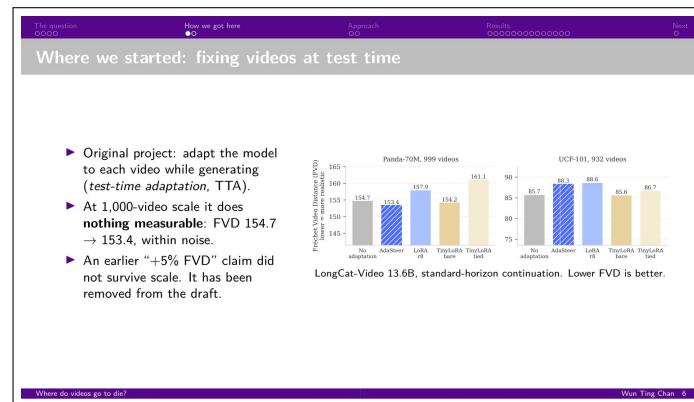
Why should anyone care beyond pretty failure pictures?

Everyone building streaming video, generative game engines or world models faces the same structural risk: the model feeds on its own output, so small errors can compound. Long rollouts often accumulate drift, and much of the recent literature is about reducing it, with anchor frames called sinks or extra training on long videos. As you'll see in a moment, it works to a large degree. Two of our three models hold up well for three minutes. So the honest framing isn't that every model collapses. It's that how a long video ends depends strongly on the model and how you run it, and we don't have a good way to describe that dependence.

That brings me to the second box. This problem is already measured, quite a lot: long-video benchmarks like VBench-Long, and recent papers that quantify colour drift, motion drift, spectral drift, representation collapse and sink collapse. What those papers measure are symptoms, one quality score at a time. What we couldn't find is a characterization of the dynamics: where trajectories converge, how fast the prompt is lost, how a small perturbation grows, and which design change moves the end state.

That's our contribution, on the right: a map of each model's end states, a memory half-life for the prompt, a sensitivity test, the finding that one design knob can flip the end state, and tools anyone can run on a new model.

Slide 6 Where we started: fixing videos at test time



Point of this slide: be honest about the starting point; the null result is what motivates the pivot.

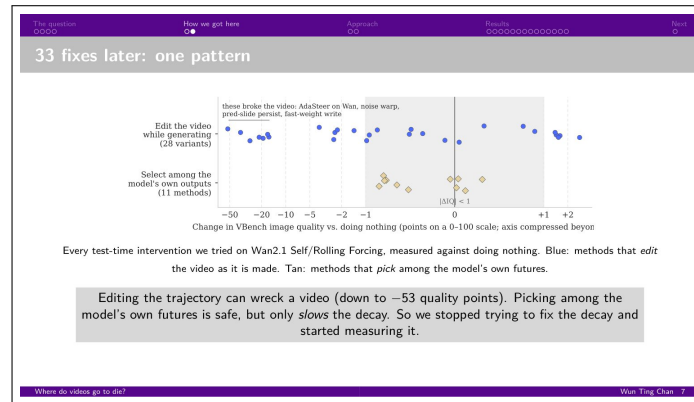
Some of you know this project under its old name, test-time adaptation. The idea was intuitive: while you generate a video, briefly fine-tune the model on that video so it stays faithful to it.

[point at the bars] This chart is the decisive experiment, and its job is to show that at scale the effect is zero. The metric is FVD, a standard measure of how realistic a set of videos looks; lower is better. Grey is doing nothing. The hatched blue bar is our method, AdaSteer. The others are standard adaptation baselines. On the left, Panda-70M with 999 videos; on the right, UCF-101 with 932.

Look at how flat it is. On Panda, doing nothing scores 154.7 and our method 153.4, a difference well inside the noise. On UCF, our method is slightly worse. The heavier baselines move things around but don’t improve anything. An earlier “five percent improvement” from smaller runs did not survive scaling up, and we’ve taken it out of the draft.

That’s a real result, but it’s a negative one. And it pushed us to ask a different question.

Slide 7 33 fixes later: one pattern



Point of this slide: the pivot. Turn a pile of failures into one principle.

We didn't stop at one method. Over the summer we tried 33 families of test-time fixes on the streaming models. This chart puts every one of them on the same ruler.

The purpose of this plot is to show a pattern across all those attempts, not to judge any single one. The horizontal axis is the change in image quality compared with doing nothing; zero is the vertical line. The axis is stretched so you can see small and catastrophic changes at once. The grey band is plus or minus one point, effectively no change.

Two kinds of method. The blue dots on the top row edit the video as it's being made: they change the noise, the weights or the memory. The tan diamonds on the bottom row select: they let the model propose several futures and keep the best one.

The edits are spread across the whole axis. Some help a little, and some destroy the video: the cluster on the far left lost 20 to 50 quality points. The selection methods all sit inside the grey band. They're safe, but in other experiments they only slow the decay; they never stop it.

That was the moment of the pivot. If every fix either breaks the video or only delays the inevitable, maybe the decay itself is the interesting object. So we stopped trying to fix it and started measuring it.

Slide 8 Setup

Setup

Models (all on Wan2.1, 1.3B)

- **Self Forcing:** rolling window, no anchor
- **Rolling Forcing:** keeps the first chunk as an anchor ("sink")
- **LongLive:** sink plus long-video tuning

Measurements, every frame

- **Content:** DINOv2 embedding (what is in the frame)
- **Look:** brightness, contrast, saturation, colourfulness, sharpness, spectral slope
- **Motion:** frame difference, optical flow

Rollouts

8 MovieGen prompts $\times$ 2 seeds $\times$ 3 models = 48 videos of 3 minutes (2,877 frames each). Plus 128 real 30s clips as a control. About 8 H200-hours so far.

Treat the generator as a dynamical system

The state is the recent frames. The rule is the model. The noise is the random seed.

Where do videos go to die? Wan Ting Chan 8

Point of this slide: just enough detail to trust the results that follow.

A quick word on the setup, so you can trust what comes next.

Three models, all built on the same 1.3-billion-parameter Wan2.1 video backbone, so differences between them come from design, not size. Self Forcing uses a rolling window and no anchor. Rolling Forcing keeps the very first chunk around as a permanent anchor, which the field calls a “sink”. LongLive also has a sink, plus extra training on long videos.

We ran 8 prompts, 2 random seeds each, 3 models: 48 videos of three minutes. As a control, we also have 128 real video clips. So far this has cost about 8 GPU-hours.

On every single frame we record three kinds of thing: what’s in it (an embedding from DINOv2, a standard image encoder), how it looks (brightness, saturation, sharpness and so on), and how much it moves (optical flow). The key reframing is the last box. We treat the generator as a dynamical system: the state is the recent frames, the rule is the model, and the noise is the seed.

Slide 9 Toolkit

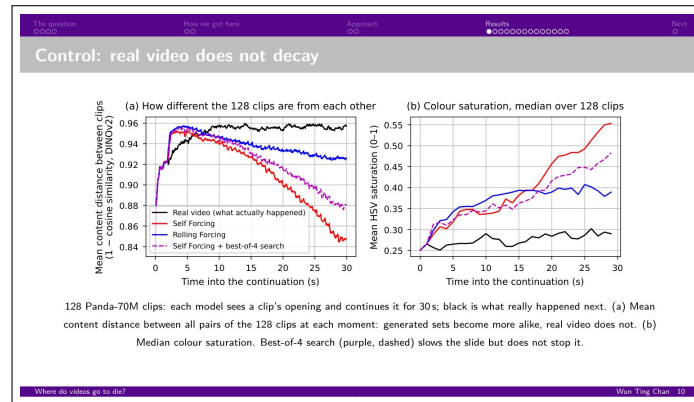
Toolkit: dynamical-systems questions, video measurements		
Concept	Question	Measurement
Attractor	Where do videos end up?	Distance between videos of different prompts; can the prompt still be identified from the frame?
Relaxation	How fast?	Exponential fit to that distance; prompt <i>half-life</i>
Lyapunov (twins)	Does a tiny nudge change the story?	Re-run with one noise block perturbed; track how fast the twins separate
Bifurcation	Which knob flips the end state?	Same weights, change sink size or attention window
Recurrence	Is the motion structured or random?	Recurrence analysis vs. phase-randomized surrogates
Early warning	Can we see collapse coming?	Rising variance and autocorrelation before collapse

Point of this slide: a roadmap for the results section; each row is a slide that follows.

This table is the roadmap for the rest of the talk. Each row takes a concept from dynamical systems, states it as a plain question, and gives the measurement we use. I'll go through them in order.

Attractor: where do videos end up? We measure how different the videos of different prompts are, and whether you can still tell which prompt a frame came from. Relaxation: how fast? We fit a decay curve and read off a half-life. Twins: does a tiny nudge change the story? We re-run a video with one tiny perturbation and watch the two copies separate. Bifurcation: is there a knob that flips the end state? Recurrence: is the motion structured or random? And early warning: can we see the collapse coming? That last one will turn out to be a “no”, and I'll show it anyway.

Slide 10 Control: real video does not decay



Point of this slide: rule out the boring explanation before claiming anything.

Before claiming that models have attractors, we need to rule out a boring explanation: maybe all video, real or generated, drifts like this over time.

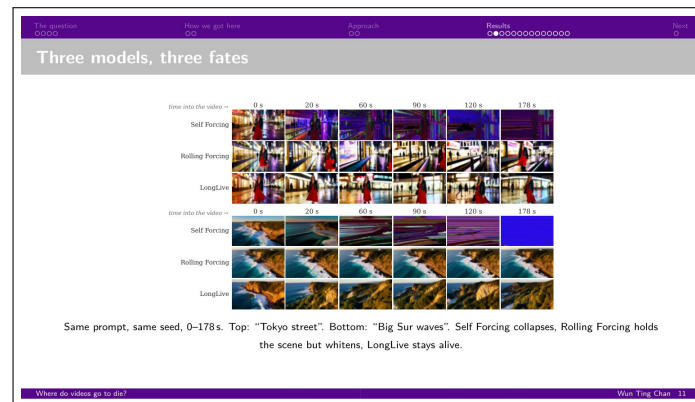
So this slide is a control. We take 128 real clips, give the models the opening and let them continue for 30 seconds, then compare against what really happened in each clip.

[left panel] The left panel measures diversity: how different the 128 videos are from each other. All curves rise in the first couple of seconds as the clips diverge from their shared start. Then they split. The black line, real video, stays flat near 0.96: real clips stay as different from each other as ever. Self Forcing, in red, falls steadily to 0.85; the 128 videos are becoming more similar to each other. Rolling Forcing in blue falls more slowly. That falling diversity is the signature of an attractor pulling everything together.

[right panel] The right panel shows one visible symptom, colour saturation. Real video, in black, stays flat. Self Forcing climbs from 0.25 to 0.55. That's the neon purple you saw on slide 3, starting within the first 30 seconds.

The dashed purple line is Self Forcing with best-of-four search, one of the safe selection methods. It's better than plain Self Forcing but still slides the same way. Searching slows the decay; it doesn't stop it.

Slide 11 Three models, three fates



Point of this slide: decay is not one phenomenon. Three models, three qualitatively different outcomes.

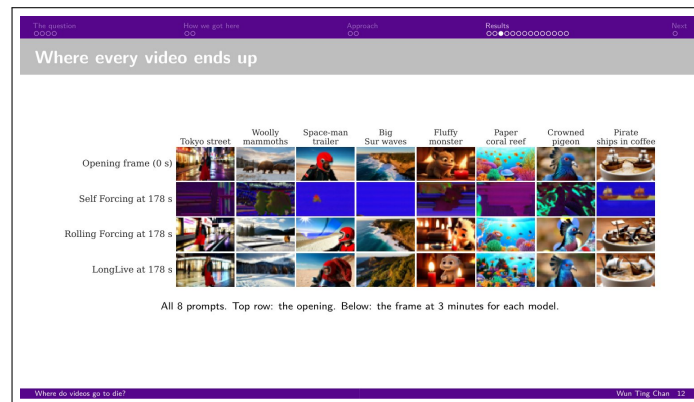
Now the three-minute runs. Same prompt and same seed, three models in each block.

[top block, Tokyo street] Self Forcing, top row, follows the path you already know: the street turns into neon streaks. Rolling Forcing, the middle row, keeps the street and the woman, but things get brighter and washed out, like glare. LongLive, the bottom row, still shows a recognisable street scene with people at three minutes.

[bottom block, Big Sur] The waves tell the same story. Self Forcing ends in a flat blue field. Rolling Forcing keeps the coastline but whitens it. LongLive is still showing waves.

So “long videos get worse” is actually three different phenomena: collapse into a shared state, slow fading in place, and staying alive. Any explanation of the decay has to account for all three.

Slide 12 Where every video ends up



Point of this slide: show the pattern holds for every prompt, not just the two we picked.

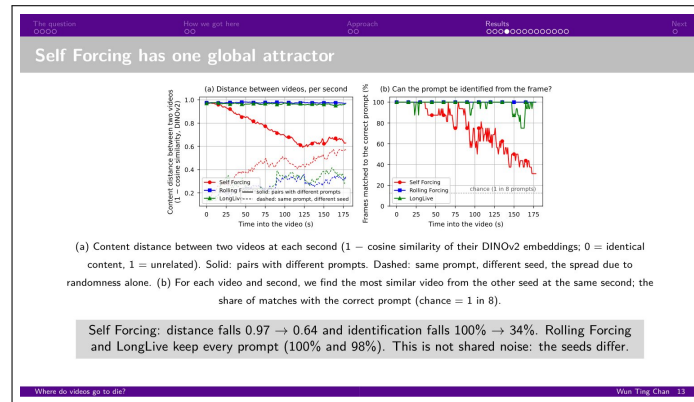
You might suspect I picked two flattering examples. So here is every prompt.

[top row] The top row is the opening frame for each of the 8 prompts. [row by row] Below it, the frame at three minutes from each model.

Read the Self Forcing row from left to right: it's essentially one image eight times, purple and blue streaks. Whatever you asked for, you end up here. The Rolling Forcing row is still eight different scenes, faded and over-bright, but you can match each one to its prompt. The LongLive row looks like eight different, living videos.

This is the visual version of the attractor claim. One model has a single global end state; the other two keep each prompt in its own basin.

Slide 13 Self Forcing has one global attractor



Point of this slide: turn the pictures into numbers, and rule out shared noise.

Pictures are convincing, but we want numbers. This slide makes the attractor claim quantitative, with two complementary measurements.

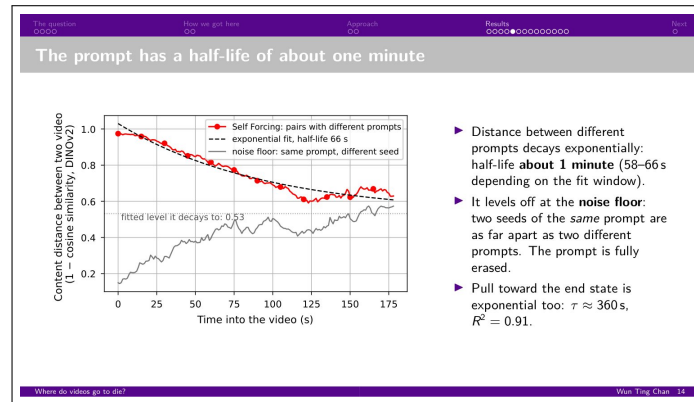
[left panel] The left panel asks: how different are videos of different prompts? The vertical axis is the content distance between two videos; 1 means completely different. Solid lines are pairs with different prompts. Rolling Forcing and LongLive sit flat near 0.97 for the full three minutes. Self Forcing starts there too, then falls steadily to about 0.64. Different prompts are literally becoming the same video.

The dashed lines are the reference. They compare two videos with the same prompt but different random seeds, the natural spread you'd expect from randomness alone. For Self Forcing, the solid and dashed lines are converging. By the end, two different prompts are about as similar as two runs of the same prompt.

[right panel] The right panel asks the same thing in a more intuitive way. If I show you a single frame, can you tell which of the 8 prompts it came from? Chance is 12.5%, the dotted line. Rolling Forcing stays at 100% and LongLive at 98%. Self Forcing falls from 100% to about a third.

One important check: these videos use different random seeds, so this isn't two copies sharing noise and drifting together. The model is pulling them together.

Slide 14 The prompt has a half-life of about one minute



Point of this slide: a single memorable number for the forgetting rate.

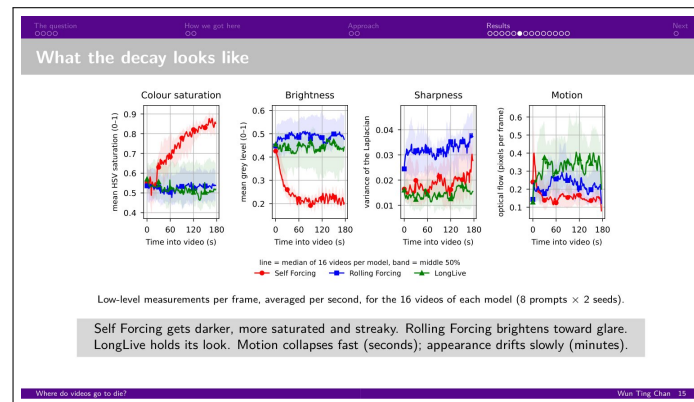
If Self Forcing forgets the prompt, how fast? This slide's job is to give that a single number.

[red line] The red line is the same Self Forcing curve as before, the distance between different prompts over time. The dashed black line is an exponential decay fitted to it, the same shape as radioactive decay. It fits well, so we can quote a half-life: about one minute. Depending on exactly which time window you fit, it comes out between 58 and 66 seconds. Every minute, the model forgets about half of what makes your prompt special.

[grey line] The grey line is the noise floor: two seeds of the same prompt. Notice where the red curve levels off: right at that floor. That means the prompt isn't just partially forgotten. By the end, knowing the prompt tells you nothing more than knowing the random seed. It's fully erased.

A second measurement, how quickly each video approaches the shared end state, is also exponential, with a time constant of about six minutes and a very clean fit ($R^2 = 0.91$). So this isn't a vague trend. It's a measurable rate.

Slide 15 What the decay looks like



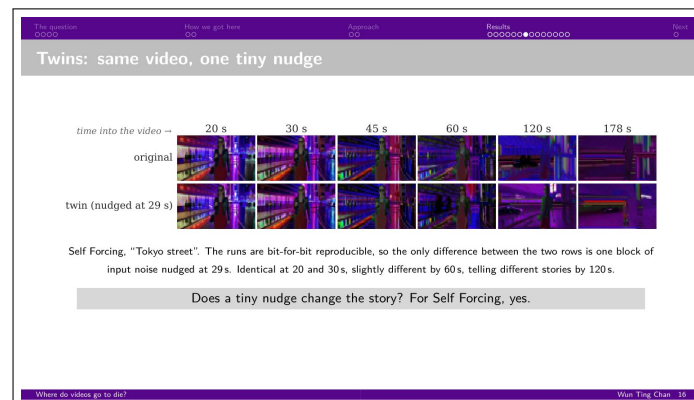
Point of this slide: connect the abstract embedding measurements back to what a viewer sees.

So far I've measured content in an abstract embedding space. This slide connects it to things a viewer would notice: colour, brightness, sharpness and motion. Each line is the median over 16 videos per model, with the band showing the spread.

[saturation] Saturation: Self Forcing, in red, climbs from about 0.55 to 0.85, oversaturated neon. The other two stay put. [brightness] Brightness: Self Forcing darkens, from 0.42 to about 0.21. Rolling Forcing, in blue, is slightly brighter than it started, which is the glare you saw. [sharpness] Sharpness: Rolling Forcing gets crisper and harsher. [motion] Motion is the interesting one. Self Forcing's motion collapses within seconds, from 0.40 to about 0.14, while its colours take minutes to drift.

So there are two clocks running: motion dies fast, appearance drifts slowly. And each model drifts in its own direction, so "decay" isn't one universal process.

Slide 16 Twins: same video, one tiny nudge

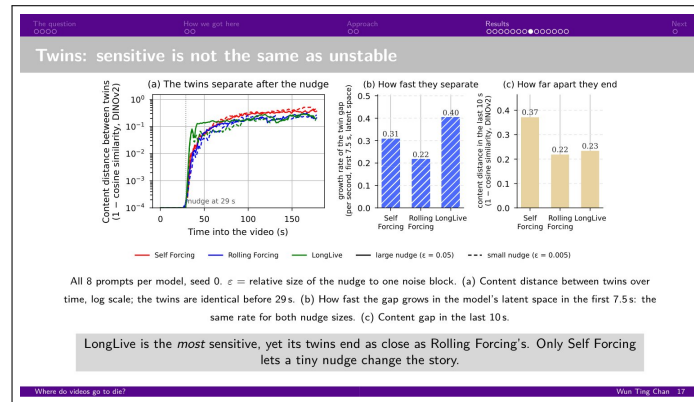


Point of this slide: set up the twin experiment visually before quantifying it.

Now a classic physics experiment: twins. These models are bit-for-bit reproducible: run the same prompt with the same seed twice and you get identical videos. So we make exactly one tiny change. At 29 seconds we nudge a single block of the random noise the model is fed, and let both copies continue.

[photo strip] Top row is the original, bottom row is the twin, both Self Forcing on the Tokyo street. At 20 and 30 seconds they're identical, as they should be. By 60 seconds there are small differences in the streaks and colours. By two minutes they're telling different stories and collapsing in different ways. For this model, a tiny nudge changes the outcome. The next slide asks whether that's true for every model.

Slide 17 Twins: sensitive is not the same as unstable



Point of this slide: separate being sensitive from being unstable; that distinction is the surprise.

This slide quantifies the twins for all three models, across all 8 prompts.

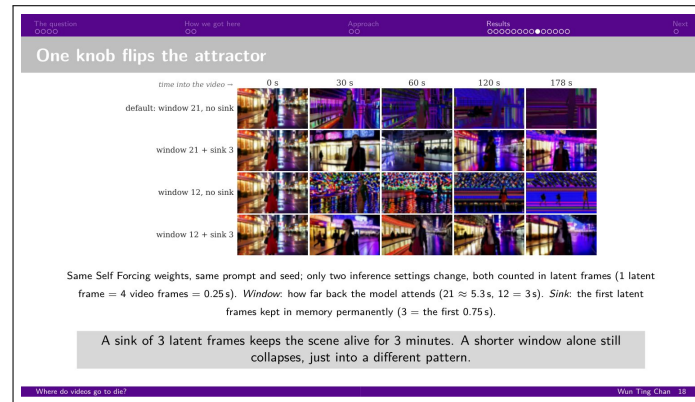
[panel a] Panel (a) tracks how far apart the twins are over time, on a log scale, using the same content distance as before. Before 29 seconds the twins are identical. That's why the lines sit on the floor of the plot: a log axis can't show zero. Right after the nudge, the gap for every model grows about thirtyfold within roughly ten seconds, then levels off. Solid lines are the larger nudge and dashed lines the ten-times smaller one, and they rise at the same rate. The growth rate is a property of the model, not of how hard you push it. That's the hallmark of what physicists call a Lyapunov exponent.

[panel b] Panel (b) gives that growth rate per model, measured in the model's own latent space over the first seven and a half seconds. LongLive is the most sensitive, at 0.40 per second, then Self Forcing at 0.31, then Rolling Forcing at 0.22.

[panel c] Panel (c) holds the surprise: how far apart the twins' content ends up, over the last ten seconds. Self Forcing's twins end furthest apart, at 0.37. LongLive's end at 0.23, no further apart than Rolling Forcing's at 0.22.

So being sensitive is not the same as being unstable. LongLive reacts fastest to small changes but keeps telling the same story. Only Self Forcing lets a tiny nudge change the plot.

Slide 18 One knob flips the attractor



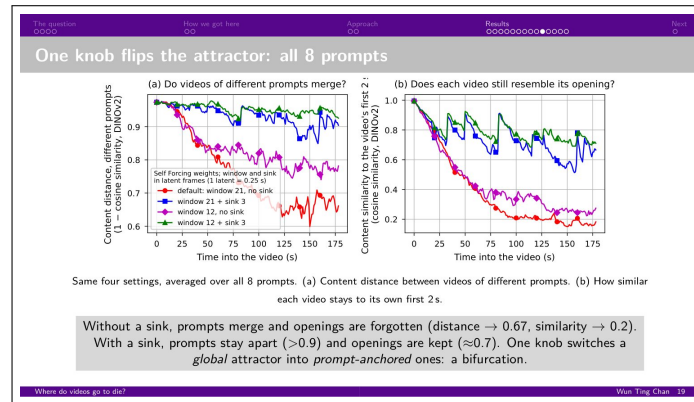
Point of this slide: the strongest result. Show it on one prompt first.

This is the result I'd most like you to remember.

So far I've compared different models, and a sceptic could say they differ in training data, training time and a dozen other things. So here we hold the weights fixed: every row is the same Self Forcing model, same prompt, same seed. We change only two settings of how it runs, both counted in what the model calls latent frames. One latent frame is four video frames, a quarter of a second.

[rows] The top row is the default: the model looks back over 21 latent frames, about five seconds, with no sink. It collapses into streaks as usual. The second row adds a sink of 3 latent frames: the first three quarters of a second of video stay in memory permanently. Look at the end: the woman and the street are still there at three minutes. The third row shortens the window to 12 latent frames, three seconds, with no sink. It still collapses, but into a different-looking state, a bright kaleidoscope instead of streaks. The fourth row does both, and it stays anchored.

Slide 19 One knob flips the attractor: all 8 prompts



Point of this slide: prove the single-prompt picture holds across prompts, and name it a bifurcation.

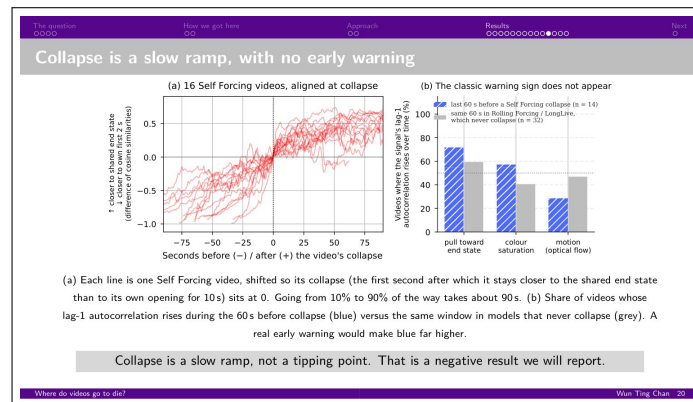
One prompt could be luck, so here are the same four settings averaged over all 8 prompts.

[panel a] Panel (a) is the distance between videos of different prompts, the measure we used to detect the global attractor. Without a sink, the red default and the purple short-window lines fall: prompts merge, ending around 0.67 for the default. With a sink, the blue and green lines stay above 0.9 for the whole three minutes: prompts stay apart.

[panel b] Panel (b) asks how similar each video stays to its own first two seconds. Without a sink, similarity falls to about 0.2: the opening is forgotten. With a sink it holds around 0.7.

The point: three quarters of a second of permanent anchor switches the system from one global attractor to separate, prompt-anchored ones. In dynamical-systems language, a small change in a parameter that changes the type of attractor is a bifurcation. It also separates the roles of the two knobs: the sink controls whether the prompts merge, and the window controls what the collapse looks like.

Slide 20 Collapse is a slow ramp, with no early warning



Point of this slide: report the negative result honestly; it is informative.

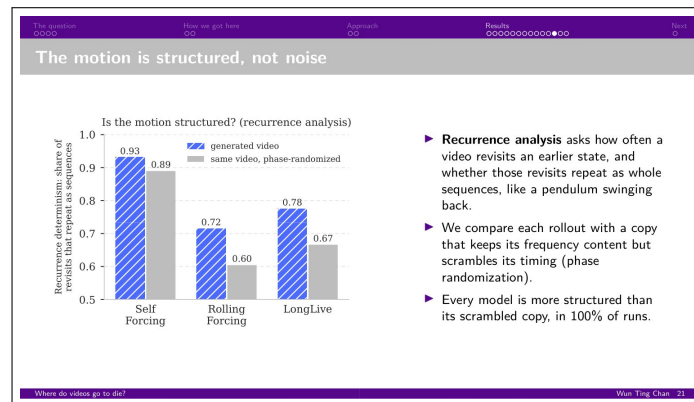
Two more tools from physics, and this one gives a “no”.

[panel a] Panel (a) asks whether collapse is a sudden tipping point or a slow slide. Each faint red line is one Self Forcing video. The vertical axis compares how similar the current frame is to the shared end state with how similar it is to the video’s own opening: above zero means it is closer to the end state. We shift each video in time so that its moment of collapse, the point after which it stays on the end-state side, sits at zero. If collapse were a tipping point, you’d see flat lines that jump at zero. Instead you see a long, smooth ramp: going from 10% to 90% of the way takes about 90 seconds.

[panel b] In ecology and climate science, tipping points are often announced by warning signs. A standard one is rising autocorrelation: the system gets sluggish and each moment looks more like the last. Panel (b) tests that. Blue is the 60 seconds before each Self Forcing collapse; grey is the same window in the other two models, which never collapse. A real warning sign would make the blue bars much taller. They aren’t: 71% against 59% for the pull toward the end state, 57 against 41 for saturation, and for motion it’s reversed. With 14 and 32 videos, none of that is a usable warning.

So the decay is a slow ramp, not a cliff. That’s a negative result and we’ll report it, because it rules out a whole class of “detect the tipping point and rescue” methods.

Slide 21 The motion is structured, not noise

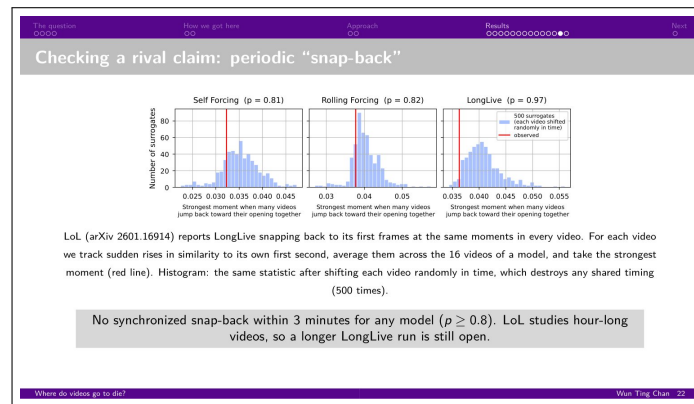


Point of this slide: a short positive check: the trajectories have real structure.

A quick positive check. Recurrence analysis asks how often a video comes back to a state it was in before, and whether those revisits repeat as whole sequences, the way a pendulum swings back through the same path. That property is called determinism.

[bars] Blue is each model's real rollout. Grey is the same rollout after phase randomization, which keeps its frequency content but scrambles the timing. For every model, the real rollouts are more structured than their scrambled copies: 0.93 against 0.89 for Self Forcing, 0.72 against 0.60 for Rolling Forcing, 0.78 against 0.67 for LongLive. That holds in 100% of runs. So these trajectories are structured dynamics, not noise, which is what justifies using dynamical-systems tools in the first place.

Slide 22 Checking a rival claim: periodic “snap-back”



Point of this slide: show we take competing findings seriously and test them.

Part of doing this properly is taking competing findings seriously. A recent paper, LoL, reported that LongLive periodically “snaps back” to its opening frames at the same moments in every video. That would contradict our claim that LongLive stays alive, so we tested it.

The logic: if every video snaps back at the same moments, then sudden jumps back toward the opening should line up across videos. For each video we track sudden rises in its similarity to its own first second, average those across the 16 videos of a model, and take the strongest moment. That’s the red line. Then we build a reference: shift each video randomly in time, which destroys any shared timing, recompute, and repeat 500 times. That’s the blue histogram, what chance alignment looks like.

[histograms] If snap-back were real, the red line would sit far to the right of the histogram. For all three models it sits inside it, with p-values of 0.8 and above. Individual jumps do happen, but at scattered times.

One caveat: LoL studied hour-long videos and we’ve only gone to three minutes. So the honest statement is “not within three minutes”, and a longer LongLive run is on our list.

Slide 23 Has anyone done this already?

The Journal of Neuroscience	Manuscript ID	Approach ID	Review Number	Date
Has anyone done this already?			00000000000000000000	Jan 2, 2024

About 60 targeted searches (2024 to Oct 2026), rechecked Oct 6. Long-horizon degradation is **measured widely**. What we did not find is any work that treats streaming video generators as **dynamical systems**.

Closest work	What it shows	Relation to ours
VBench-Long, Relax Forcing, In-Distribution Forcing (2610.03120) LoL (2601.16514)	Quality, consistency, colour and motion drift over minutes Sink-collapse loops at hour scale	measure symptoms; we model dynamics partial: test longer
Deep / Rolling Sink, FreqForcing (2607.27110) Effective-rank collapse (2607.27036) Meta-ARVDM (2503.10704) Birol et al., Nat. Commun. 2024 Hintze et al., Patterns 2025	A sink slows drift; drift as spectral-energy shift Abrupt representation collapse at drift onset Theory of history forgetting Attractor-like regimes inside the denoising steps Iterated image loops collapse to ~12 attractors	partial: we show a type switch reconcile with "slow ramp" frames, not the prompt other time axis concept, not video

Safe claim: existing work measures long-horizon degradation and specific failure modes; we characterize where trajectories converge, how fast prompt information is lost, how perturbations grow, and which inference settings change the attractor. Avoid: "nobody has measured it", "every model collapses", "sinks prevent drift" (known).

Where do videos go to die?
Wang, Tan, Chan

Point of this slide: novelty, stated precisely and defensibly.

The natural question is whether this has been done. We ran about 60 targeted literature searches, covering 2024 up to this week, and rechecked them today.

Be precise about the answer. Long-horizon degradation is measured widely. VBench-Long scores quality and consistency over long videos. Papers from the last few months report colour and motion drift over minutes, spectral drift, a collapse in the model’s internal representations at drift onset, and the sink collapse I just tested. What we did not find is anyone treating these streaming generators as dynamical systems: asking where trajectories converge, how fast the prompt is lost, how perturbations grow, and which settings change the attractor.

[table] The table lists the closest work and how it relates to ours. Several sink papers already show that a sink slows drift, so our claim is the stronger one: it changes the type of attractor. The representation-collapse paper finds something abrupt where we see a slow ramp, and reconciling the two is a to-do. There is also attractor language elsewhere. Biroli and colleagues in Nature Communications describe attractor-like regimes inside the denoising steps that generate a single image. That’s a different time axis from ours, which is time within the video. And looping image models are known to collapse to about a dozen attractors.

[grey box] So here is the claim we'll make in the paper: existing work measures degradation and specific failure modes; we characterize the dynamics. What we won't claim: that nobody has measured long-video decay, that every model collapses, or that sinks prevent drift. All three would be easy to knock down.

Slide 24 Next two weeks

Next two weeks

Scale and sharpen (about 10 H200-hours)

- ▶ 8 → 32 prompts for every result above
- ▶ Sink sizes 1 and 2: where exactly does the switch happen?
- ▶ Rolling Forcing *without* its sink: does it collapse like Self Forcing?
- ▶ One long LongLive run (beyond 3 min) for the snap-back test

Paper (CVPR 2027, deadline Nov 16)

- ▶ Lead with the attractor map and the one-knob switch
- ▶ Supporting chapters: the TTA null at scale and the 33-intervention atlas
- ▶ Release the toolkit as a drop-in diagnostic

Takeaway. Long videos do not just "get worse". Each model falls into its own kind of attractor, at a measurable rate, and a single design knob can change which kind.

Where do videos go to die? What You Can 24

Point of this slide: a concrete plan and one sentence to take home.

Where this goes next.

On the left, scaling and sharpening, about 10 more GPU-hours. Every result you saw uses 8 prompts; we'll go to 32 so the statistics hold up. We'll try sink sizes of 1 and 2 to find exactly where the switch happens. We'll remove Rolling Forcing's sink to see whether it then collapses like Self Forcing, a direct test of the bifurcation claim on a second model. And we'll do one long LongLive run to settle the snap-back question.

On the right, the paper, aimed at CVPR with a November 16 deadline. It leads with the attractor map and the one-knob switch. The test-time adaptation null and the 33-method atlas become supporting chapters: they're why selection only slows the decay and editing breaks it. And we'll release the measurements as a drop-in diagnostic for anyone with a new model.

[final box] If you remember one sentence: long videos don't just "get worse". Each model falls into its own kind of attractor, at a rate we can measure, and a single design knob can change which kind.

Thank you. Happy to take questions.